

Speaker 2 of 5 — Project Technology Stack

Career Compass SG · Module 1 Assignment · Group 7

Your slot: 2:15 – 3:45 (1 minute 30 seconds) · You are the shortest slot — stay tight

Your job in one sentence

Show that the architecture was a decision, not an accident. Anyone can call `pd.read_csv`. Your 90 seconds are about *why* the pipeline has three stages and what that bought us — a dashboard that filters a million rows in six hundredths of a second.

Where you sit in the flow

	Speaker	Topic
	1	Dashboard overview
→ YOU	2	Technology stack
	3	Data cleaning
	4	Data analysis 1 — market & pay
	5	Data analysis 2 — competition, skills & scoring

Speaker 1 hands you: "...now, how do you make a million rows feel instant?" You hand to Speaker 3 with: "That's the machinery. But none of it matters if the numbers going in are wrong — and this dataset had a trap in it."

Script

Part A — The stack and the three stages (2:15 – 3:00)

(Slide: the pipeline diagram)

Python and pandas, in three stages, so the dashboard never touches the raw CSV.

SGJobData.csv (273 MB, 1.05M rows) ↓ src/etl_clean.py chunked read, cleaning, features ~15 s 3 Parquet tables ↓ src/build_aggregates.py summaries + Career Fit Score ~2 s 8 aggregate tables + KPI JSON ↓ app/ (Streamlit + Plotly) reads Parquet, filters live <0.1 s

Stage one reads 273 megabytes in **six chunks of 200,000 rows**, cleans each, then applies the two rules that need the whole file — de-duplication and outlier capping. **Fifteen seconds**.

Stage two pre-computes every summary the dashboard draws.

Stage three is Streamlit and Plotly — Streamlit because it's Python end to end, Plotly because every chart needed hover tooltips. The notebook and report figures use matplotlib and seaborn.

Part B — The number that justifies all of it (3:00 – 3:30)

(Slide: the performance figures)

Here's what that bought us. Our working table is **1.77 million rows**. As **Parquet with categorical dtypes** it sits in **170 megabytes**, and a full **filter-and-group-by takes 0.06 seconds**.

That speed is why our filters are real. Many dashboards filter a frozen summary table — you move a slider and the same numbers come back reshuffled. Ours recomputes from row level every time, across seven pages.

Part C — One engineering decision worth naming (3:30 – 3:45)

One decision I'd defend: **the dashboard imports its scoring functions from the pipeline** rather than reimplementing them, so the numbers on screen and in our report **cannot drift apart**.

That's the machinery. None of it matters if the numbers going in are wrong — and this dataset had a trap.

Numbers you must know cold

Figure	Value
Raw file	273 MB, 1,048,585 rows
Chunk size	200,000 rows × 6 chunks
ETL runtime	~15 seconds (aggregates: ~2 s more)
Working table in the app	1,767,829 rows / 170 MB in memory
Filter + group-by cycle	0.06 seconds
Dashboard pages / chart types	7 pages, ~17 chart types

The stack, one line each

Tool	Why it is there
<code>pandas</code>	the whole cleaning and aggregation layer
<code>PyArrow / Parquet</code>	columnar storage + categorical dtypes — the 170 MB figure
<code>Streamlit</code>	Python end to end, multi-page routing for free, <code>st.session_state</code> shares filters across pages
<code>Plotly</code>	interactive charts with hover tooltips in the app
<code>matplotlib + seaborn</code>	static figures for the notebook and report
<code>@st.cache_data</code>	caches loads and aggregations keyed on the filter signature

Questions you own

"Why Parquet and not just keep the CSV?"

Columnar, compressed and typed. Reading the columns we need from Parquet is roughly 20× faster than re-parsing 273 MB of text, and categorical dtypes cut the string columns from hundreds of megabytes to tens.

"Why not a database — SQL, DuckDB?"

Nothing here needs a query engine. It's one dataset, refreshed in batch, and pandas already does the whole job in 15 seconds. A database would add a moving part without removing one.

"Why Streamlit over Power BI or Tableau?"

Our cleaning rules and the scoring model are Python. Streamlit lets the dashboard call that code directly. In a BI tool we'd have had to reimplement the score in the tool's own expression language — which is exactly the drift problem I just described.

"Is `@st.cache_data` hiding stale data?"

No — the cache key includes the filter signature, so changing any filter recomputes. The underlying Parquet only changes when we re-run the ETL.

"How would this scale to a real product?"

The pipeline already handles the full million in 15 seconds and the app reads pre-built Parquet. The missing pieces are a scheduled refresh, user accounts and hosting — not the analytics.

Rehearsal checklist

- [] Timed at **1:30 or under** — this is the tightest slot in the presentation
- [] You can say "1.77 million rows, 170 megabytes, 0.06 seconds" without reading it
- [] You know what a categorical dtype does, in one sentence, if asked
- [] Handover line to Speaker 3 practised — it sets up their whole section